



**BSC-Cybersecurity
Academic Year 2025-26**

Shamsudin Aminullah – P2771339

FYP First Deliverable – CSEC 3100D

Project Title:

Phishing Email Detection

Supervisor:

Dunja Majstorovic

Table of Contents

<i>Literature Review</i>	3
Introduction to the Phishing Threat Landscape	3
Evolution and Categorization of Phishing Techniques	3
The Role of Social Engineering	4
Conventional Defense Mechanisms and Their Limitations	5
Advanced and Data-Driven Detection Strategies	5
<i>Functional Requirements</i>	6
<i>Indicative Test Plan</i>	8
<i>System Design Documentation</i>	10
<i>Implementation Report</i>	14
References	15

Literature Review

Introduction to the Phishing Threat Landscape

E-mail is one of the most highly used communication channels in organizations today, making it a common target for cyber attackers. Even with continuous improvements in network security, the human element often remains the biggest weakness. Phishing has become a major type of cyberattack, relying heavily on misleading messages and social engineering tricks to fool the person receiving the email. In many studies, it is noted that the number of phishing attacks has been growing steadily to reach a high level, with email being a favorite hacking method because of the affordability, scalability, and potential ability to compromise human trust, as opposed to technical weaknesses (Verizon, 2023; APWG, 2022).

The main goal of a phishing attack is to pretend to be a trusted source, tricking users into bypassing normal security measures. These attacks are no longer just simple, widespread spam; they have grown into highly targeted operations that cause serious problems for businesses, including stolen passwords, unauthorized access, lost data, and financial theft. As attackers constantly improve their methods using language tricks, misleading URLs, and harmful attachments that look like normal files organizations urgently need better detection systems.

This review will analyze how modern phishing attacks work, look at the weaknesses of current defense methods, and explore advanced, data-driven techniques that can help spot new threats.

Evolution and Categorization of Phishing Techniques

Historically, phishing campaigns relied on volume rather than quality. Early attacks used a "spray-and-pray" approach, sending out millions of generic emails such as fake bank alerts or lottery winning shopping a small percentage of people would click the links. However, as standard spam filters improved and users became more aware of these basic tricks, attackers had to adapt. Today, the focus has shifted from mass-mailing to highly targeted, customized attacks that are much harder for traditional security systems to catch (APWG, 2022; Trend Micro, 2021).

To understand how to build a better detection system, it is important to look at the specific categories of modern phishing:

- **Spear Phishing:** Unlike mass phishing, spear phishing targets a specific individual or organization (Parsons et al., 2015). Attackers gather background information on their target often from social media or corporate websites to make the email look like it comes from a known and trusted sender, such as a colleague or a specific vendor. Because these emails are highly personalized, they rarely trigger standard spam filters.
- **Whaling:** This is a more focused version of spear phishing that specifically targets high-level executives, such as CEOs or Chief Financial Officers (CFOs) (Almeida et al., 2020). The goal of a whaling attack is usually to steal highly sensitive corporate data or

trick the executive into authorizing large wire transfers. These emails are crafted with extreme care and often use legal or business language to create a sense of urgency.

- **Clone Phishing:** In this technique, an attacker takes a legitimate, previously delivered email that contains a link or an attachment. They create a nearly identical copy (a clone) but replace the safe link or file with a malicious one which makes it more difficult to detect because of the high rate of resemblance to trusted messages (Jakobsson & Myers, 2007). The email is then sent from an address that looks almost exactly like the original sender, often with a message saying it is an "updated" version of the previous email. This method relies heavily on the trust the user already has in the original communication.

The shift toward these customized attacks shows exactly why simple keyword matching is no longer enough. Because attackers now spoof trusted senders and use well-written text, modern detection requires a deeper look into the technical features of the email, such as hidden URL routing, attachment types, and header anomalies.

The Role of Social Engineering

While the technical delivery of a phishing email is important, the actual success of the attack relies heavily on social engineering. Social engineering is the practice of manipulating people into performing actions or giving away secret information. In the context of email security, attackers use these tricks to bypass a person's natural caution and common sense (Dhamija et al., 2006).

Attackers design their emails to trigger specific emotional responses that force the user to make quick, unthinking decisions. The most common psychological triggers include:

- **Urgency and Fear:** Many phishing emails claim that an account will be closed, a payment has failed, or a security breach has just happened. This rush forces the user to act quickly without taking the time to double-check the sender's details.
- **Authority:** Emails are often forged to look like they come from a manager, a CEO, or a trusted organization like a bank or an IT department. People are naturally trained to follow instructions from figures of authority, making them less likely to question the request (Jakobsson and Myers, 2007; Vishwanath et al., 2011).
- **Curiosity and Greed:** Offering fake rewards, unexpected package delivery updates, or "highly confidential" corporate documents tempts users into clicking harmful links or opening infected attachments.

To make these emotional triggers work, attackers use specific writing tactics, often called linguistic manipulation. They copy standard business phrases, match the exact tone of a specific company, or personalize the greeting using background information found online. By combining these language skills with visual tricks such as stealing company logos and copying standard email layouts the attacker creates a highly convincing message.

Understanding these social engineering tactics is a key part of building a better detection tool. A modern system cannot rely only on checking the sender's technical routing; it must also be designed to spot the common text patterns and warning signs of human manipulation.

Conventional Defense Mechanisms and Their Limitations

To fight email threats, organizations have relied on standard security tools for many years. The most common tools include traditional filtering schemes and signature-based applications. These systems act as gatekeepers, scanning incoming messages to block known threats before they can reach the user's inbox.

Standard spam filters usually work by checking the sender's address against a list of blocked sources or by looking for specific spam keywords. Signature-based systems go a step further by matching the email's attachments or code against a database of known harmful files. While these methods are good at stopping basic, bulk spam, they have a major weakness: they often fail to identify new phishing attacks. Because these older systems rely on matching known data, they are blind to threats they have not seen before (Fette et al., 2007).

This creates a serious problem when dealing with modern attacks. Today's attackers constantly change their methods to slip past these defenses. For example, when attackers send personalized emails, the messages do not contain the typical keywords that trigger standard spam alarms. Furthermore, if an attacker uses domain spoofing making their email address look exactly like a trusted company or links to well-constructed fraudulent websites, traditional filters will usually let the email pass straight through.

Relying only on these older methods leaves organizations unprotected against smart, targeted attacks. To stop these new threats, security systems need to move beyond simple checklists. They must start analyzing the deeper features of every email to catch new and hidden patterns (Abu-Nimeh et al., 2007).

Advanced and Data-Driven Detection Strategies

Because traditional filters struggle to stop modern, targeted attacks, security systems must change how they evaluate emails. Instead of just checking a message against a set list of known bad addresses or files, advanced systems use data-driven analysis. This involves breaking the email down into its core parts to find hidden signs of an attack.

This process is known as feature extraction. By pulling specific pieces of data from an email file, a detection tool can examine the message much like a human security expert would. The most important features to analyze include:

- **Email Headers and Sender Details:** Attackers often fake the "From" address. Analyzing the hidden routing details in the email header can reveal if the message actually came from the server it claims to be from.
- **URL Structures:** Phishing emails usually contain links to fake websites. A detection system can look at the format of the URL, checking for slight spelling changes, hidden redirects, or unusual domain names (Sahingoz et al., 2019).

- **Attachments:** Attackers frequently hide harmful code inside normal-looking files, like PDFs or Word documents. Advanced tools inspect these files for hidden risks rather than just checking the file name.
- **Message Content:** By analyzing the actual text of the email, systems can spot the common social engineering tricks discussed earlier, such as a false sense of urgency or unusual requests for sensitive data.

Once these features are extracted, machine learning models can be used to make sense of the data. Instead of following strict, pre-written rules, a machine learning system is trained on thousands of examples of both safe and malicious emails. The model learns to recognize the subtle, complex patterns that define a phishing attack. This allows the system to accurately catch new, never-before-seen threats based on their features, rather than relying on outdated blocklists

Functional Requirements

This section outlines the specific behaviors, capabilities, and user interactions required for the Phishing Email Detection system to successfully identify and flag malicious communications.

1. System Users (Actors)

To ensure the system is practical for real-world application, it is designed to support two primary types of users:

- **Standard User:** An everyday email user (e.g., an employee) who needs a simple, fast way to verify if a suspicious email they received is safe or a phishing attempt.
- **Security Analyst / Administrator:** A technical user who requires detailed insights into *why* an email was flagged (e.g., viewing specific header anomalies, extracted deceptive URLs, or social engineering text cues) to monitor and mitigate broader network threats.

2. Core Functional Requirements

The system must successfully execute the following functions to meet the project objectives:

- **FR1: Email File Ingestion:** The system must allow users to easily input or upload standard email files (e.g., *.eml* or *.msg* formats) for analysis.
- **FR2: Automated Feature Extraction:** The system must automatically parse the uploaded email file and extract the critical components required for analysis, specifically: sender headers, embedded URLs, attachment metadata, and message body text.

- **FR3: Threat Classification:** The system must process the extracted features through a detection model (utilizing machine learning or text-analysis rules) to classify the email as either "Legitimate" or "Phishing".
- **FR4: Indicator Highlighting:** If an email is classified as a threat, the system must output results that explicitly identify the phishing red flags (e.g., a spoofed sender domain, a mismatched "Reply-To" address, or linguistic manipulation in the body).
- **FR5: Result Output:** The system must display a clear, accessible summary of the scan results to the user upon completion.

3. Use Cases

The following use cases define how the actors will interact with the system to achieve their goals.

Use Case 1: Scan a Suspicious Email

- **Primary Actor:** Standard User
- **Goal:** To determine if a received email is safe to interact with or if it is a phishing threat.
- **Main Success Scenario:**
 1. The user uploads a suspicious email file to the detection tool.
 2. The system parses the file and extracts the relevant features (headers, URLs, text).
 3. The system evaluates the features using the detection model.
 4. The system successfully outputs a clear "Safe" or "Phishing Alert" verdict to the user.
- **Alternate Flow (Invalid File):** If the user uploads an unsupported file type, the system halts the scan and displays an error message prompting the user to upload a valid email file format.

Use Case 2: Review Detailed Threat Indicators

- **Primary Actor:** Security Analyst
- **Goal:** To understand the specific mechanisms and tactics used in a flagged phishing email to improve broader organizational defenses.
- **Main Success Scenario:**
 1. The analyst selects an email that the system has already classified as "Phishing."
 2. The system displays a detailed breakdown of the extracted features.
 3. The system visually highlights the specific anomalies that triggered the alert, such as deceptive URL routing or malicious attachment signatures.
 4. The analyst uses this detailed output to update network blocklists or security training materials.

Indicative Test Plan

1. Testing Strategy

The primary objective of this testing strategy is to ensure that the Phishing Email Detection tool is reliable, accurate, and user-friendly. The testing will verify that the system successfully meets all functional requirements outlined in the previous phase, from file ingestion to threat classification.

Testing Scope and Levels:

- **Unit Testing:** Individual components of the software will be tested in isolation. This includes testing the file parser to ensure it can correctly open *.eml* and *.msg* files, and testing the feature extraction scripts to verify they accurately pull URLs, header data, and body text without crashing.
- **Integration Testing:** This phase will test how well the different components work together. Specifically, it will verify that the extracted features are correctly passed into the machine learning detection model and that the model returns a valid classification output.
- **System Testing:** The tool will be tested as a complete, integrated system to ensure the user interface correctly displays the analysis results and highlights the relevant phishing indicators.

Test Data: Testing will utilize a pre-compiled dataset of explicitly labeled "Phishing" and "Legitimate" emails sourced from public repositories. This ensures the model is evaluated against real-world data and allows for the calculation of accuracy, false-positive, and false-negative rates.

Testing Approach: A combination of black-box testing (testing the user interface and overall outputs without looking at the code) and white-box testing (verifying the internal logic of the feature extraction and classification algorithms) will be employed.

2. Test Cases

Test Case 01: Valid Email File Upload

- **Description:** Verify that the system accepts standard email file formats.
- **Pre-conditions:** The application is running, and the user is on the main upload screen.
- **Test Steps:** 1. Click the "Upload" button. 2. Select a valid *.eml* file. 3. Click "Submit".
- **Expected Result:** The system successfully loads the file and displays a "Processing..." message.

- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

Test Case 02: Invalid File Type Rejection

- **Description:** Verify that the system rejects unsupported file formats.
- **Pre-conditions:** The application is running.
- **Test Steps:** 1. Click the "Upload" button. 2. Select a *.jpg* or *.exe* file. 3. Click "Submit".
- **Expected Result:** The system blocks the upload and displays an error message stating "Unsupported file format. Please upload an email file."
- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

Test Case 03: Feature Extraction Accuracy (URLs)

- **Description:** Verify that the system correctly extracts all embedded URLs from the email body.
- **Pre-conditions:** A test email containing three known URLs is uploaded.
- **Test Steps:** 1. Upload the specific test email. 2. Trigger the extraction process. 3. View the system's internal extraction log.
- **Expected Result:** The log clearly lists all three URLs exactly as they appear in the email's HTML/text body.
- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

Test Case 04: True Positive Detection (Phishing Email)

- **Description:** Verify that the machine learning model correctly flags a known malicious email.
- **Pre-conditions:** A labeled phishing email from the test dataset is selected.
- **Test Steps:** 1. Upload the phishing email. 2. Wait for the analysis to complete.
- **Expected Result:** The system outputs a "Phishing Alert" classification and highlights the deceptive features.
- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

Test Case 05: True Negative Detection (Legitimate Email)

- **Description:** Verify that the model correctly identifies a safe, standard business email without triggering a false alarm.
- **Pre-conditions:** A labeled legitimate email from the test dataset is selected.
- **Test Steps:** 1. Upload the legitimate email. 2. Wait for the analysis to complete.
- **Expected Result:** The system outputs a "Safe" classification with no phishing indicators flagged.
- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

Test Case 06: Threat Indicator Display

- **Description:** Verify that the UI clearly displays the reasons why an email was flagged.
- **Pre-conditions:** An email with a spoofed header and a malicious attachment is scanned and flagged.
- **Test Steps:** 1. View the final result screen for the flagged email.
- **Expected Result:** The screen contains a visual warning and specifically lists "Spoofed Sender Domain" and "Suspicious Attachment" in the summary report.
- **Actual Result:** [To be completed during execution]
- **Pass/Fail:** []

System Design Documentation

1. High-Level System Architecture

The Phishing Email Detection tool follows a modular, three-tier architecture to ensure fast processing and accurate analysis.

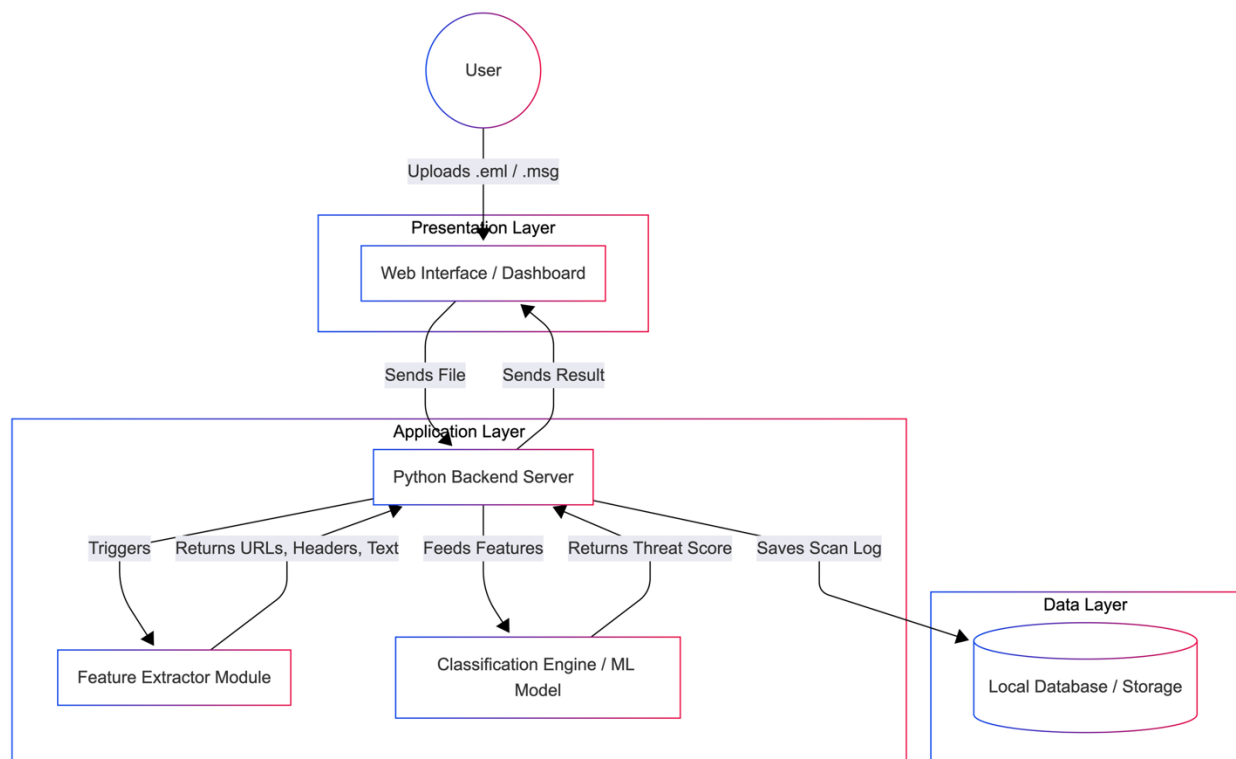


Figure 1: Architecture Of the Phishing email detection tool.

- **Presentation Layer (User Interface):** A web-based dashboard where the user uploads the .eml or .msg file and views the final analysis report.

- **Application Layer (Backend Processing):** The core engine developed in Python. It contains two main modules:
 - **Feature Extractor:** Parses the email to pull out headers, URLs, and body text.
 - **Classification Engine:** Feeds the extracted features into the machine learning model to determine the threat level.
- **Data Layer:** A lightweight database that temporarily stores the extracted features and the final classification result for the duration of the session, allowing the user to view detailed reports.

2. UML Diagrams

A. Sequence Diagram: Email Scanning Process This diagram maps the step-by-step chronological interactions between the system components when a user scans an email.

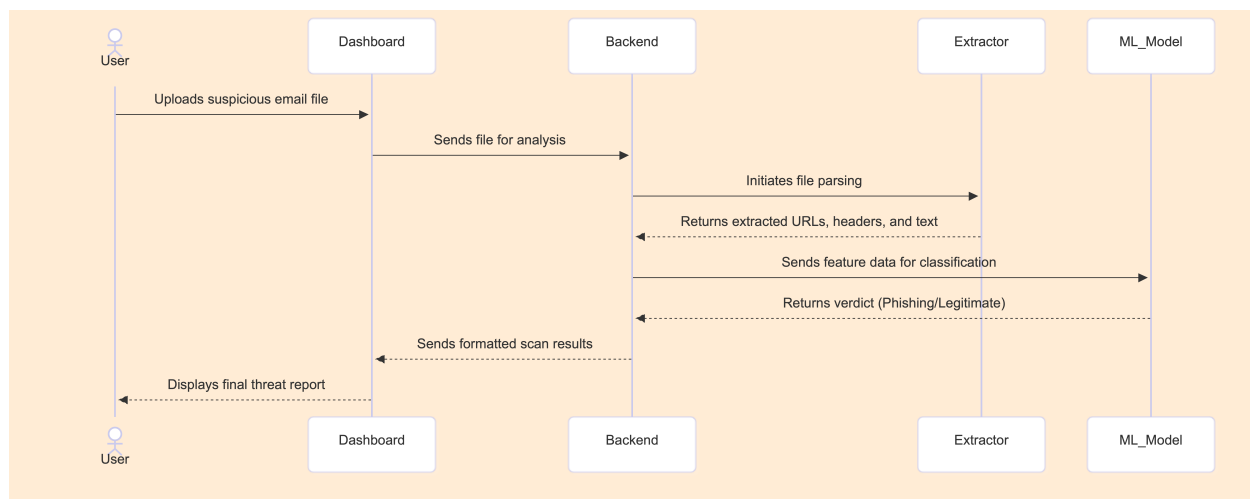


Figure 2: Email Scanning process and component interactions.

1. **User -> UI Dashboard:** Uploads suspicious email file.
2. **UI Dashboard -> Backend Server:** Sends the file for analysis.
3. **Backend Server -> Feature Extractor:** Initiates file parsing.
4. **Feature Extractor -> Backend Server:** Returns extracted URLs, headers, and text.
5. **Backend Server -> ML Model:** Sends feature data for classification.
6. **ML Model -> Backend Server:** Returns verdict (Phishing/Legitimate) and confidence score.
7. **Backend Server -> UI Dashboard:** Sends formatted scan results.
8. **UI Dashboard -> User:** Displays the final threat report.

B. Activity Diagram: System Logic Flow This flowchart visualizes the decision-making process of the tool.

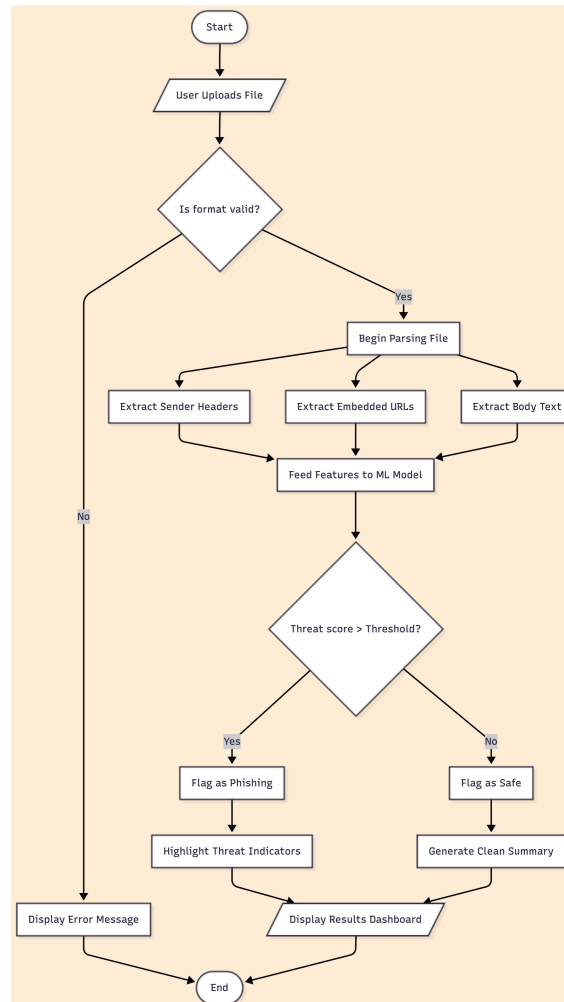


Figure 3: Systems logic flow and decision making process.

- **Start** -> Upload Email File.
- **Decision Node:** Is the file format valid?
 - *No*: Show Error Message -> **End**.
 - *Yes*: Proceed to Parsing.
- **Process Nodes (Parallel):** Extract Headers | Extract URLs | Extract Body Text.
- **Process Node:** Run features through the Classification Model.
- **Decision Node:** Is the threat score above the phishing threshold?
 - *Yes*: Flag as "Phishing" and highlight indicators.
 - *No*: Flag as "Safe".
- **Process Node:** Generate Results Summary.
- **End** -> Display Summary to User.

3. Entity-Relationship Diagram (ERD)

Even though this is primarily a processing tool, an ERD is necessary if the system logs scans for security analysts to review later.

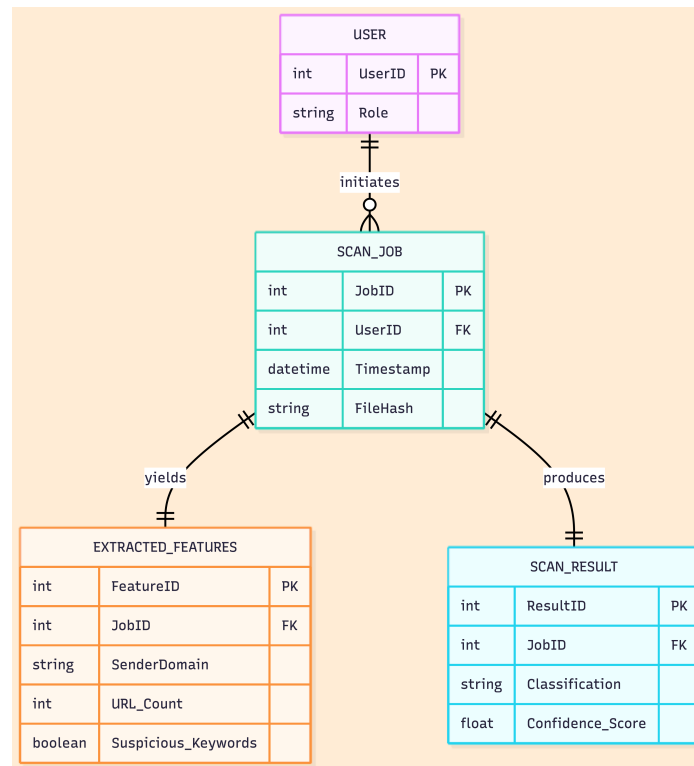


Figure 4: Entity Relationship Diagram (ERD).

- **Entity: User**
 - *UserID* (Primary Key)
 - *Role* (Standard or Admin)
- **Entity: Scan_Job** (One User can have Many Scan_Jobs)
 - *JobID* (Primary Key)
 - *UserID* (Foreign Key)
 - *Timestamp*
 - *FileHash*
- **Entity: Extracted_Features** (One Scan_Job has One set of Extracted_Features)
 - *FeatureID* (Primary Key)
 - *JobID* (Foreign Key)
 - *SenderDomain*
 - *URL_Count*
 - *Suspicious_Keywords_Found*
- **Entity: Scan_Result** (One Scan_Job has One Scan_Result)
 - *ResultID* (Primary Key)
 - *JobID* (Foreign Key)
 - *Classification* (Phishing / Legitimate)
 - *Confidence_Score*

Implementation Report

- The prototype demonstration will showcase the core functionality and practical application of the phishing email detection tool. It will provide a complete, step-by-step walkthrough of both the front-end user interface and the back-end processing logic. The primary goal of this demonstration is to prove that the system can successfully ingest a standard email file, extract its underlying features, and accurately classify it as either safe or malicious using data-driven analysis.
- During the demonstration, the front-end presentation will focus heavily on usability and the overall user experience. It will begin by showing the web-based dashboard, illustrating how a user can easily interact with the system. The demonstration will walk through the file upload process, showing how the system accepts standard email file formats, such as `.eml` and `.msg`. It will also show the error-handling capabilities of the interface, such as the warning messages displayed if a user attempts to upload an unsupported file type.
- Once a valid file is uploaded, the front-end demonstration will move to the results screen. This section will highlight how the system presents its final analysis to the user. It will show the clear visual indicators used to display a "Safe" or "Phishing" verdict. Furthermore, the demonstration will emphasize how the user interface breaks down the specific threat indicators. It will show how the dashboard explicitly lists the reasons for a positive detection, such as highlighting spoofed sender domains, listing suspicious embedded URLs, and flagging unusual text patterns. This will demonstrate that the tool is not only a background scanner but also an informative resource for the user.
- The back-end portion of the demonstration will reveal the technical mechanics and code architecture that make the detection possible. It will show the core processing engine in action. First, it will demonstrate the automated feature extraction process, proving how the back-end scripts parse the uploaded email file. It will show exactly how the system separates the header routing data, pulls out the embedded links, and reads the message body text without losing important context.
- Next, the back-end demonstration will showcase the classification engine. It will explain how the extracted features are structured and fed into the machine learning detection model. The presentation will briefly cover the logic the model uses to evaluate the data against its training set, ultimately generating a final threat score and confidence level to determine the email's safety.
- Finally, the demonstration will include a brief review of the actual codebase. Specific code snippets will be shown to highlight key programming achievements, such as the specific script used for URL extraction or the function that connects the front-end dashboard to the back-end model. The demonstration will emphasize how the code is organized into modular, readable files to ensure the system runs efficiently. Additionally, the code shown will feature clear comments and logical structuring. This final part of the presentation will prove that the tool is not only functional but also built using reliable, maintainable software development practices

References

1. Verizon. (2023). Data Breach Investigations Report (DBIR).
2. APWG. (2022). Phishing Activity Trends Report. Anti-Phishing Working Group.
3. Trend Micro. (2021). Email Threat Landscape Report.
4. Parsons, K., McCormac, A., Butavicius, M., Pattinson, M., & Jerram, C. (2015). Determining employee awareness using the human aspects of information security questionnaire. *Computers & Security*, 42, 165–176.
5. Almeida, F., Cruz, A., & Simões, P. (2020). Phishing detection through machine learning techniques. *Procedia Computer Science*, 164, 195–202.
6. Jakobsson, M., & Myers, S. (2007). *Phishing and Countermeasures: Understanding the Increasing Problem of Electronic Identity Theft*. Wiley.
7. Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. *Proceedings of the 16th International World Wide Web Conference (WWW)*, 649–656.
8. Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *Proceedings of the Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit*, 60–69.
9. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.
10. Dhamija, R., Tygar, J. D., & Hearst, M. (2006). Why phishing works. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 581–590.